DEFINITION An *action* (or *action method*) is a method that runs in response to a request. An *MVC controller* is a class that contains one or more logically grouped action methods.

Each of the three action methods calls `View()` and returns the result. This returns a `ViewResult`, which instructs the MVC framework to render a Razor view for the action. You'll learn more about this process in section 19.4. The `Error` action method also sets an object in the call to `View()`. This is the view model, which is passed to the Razor view when it's rendered.

NOTE MVC controllers use explicit *view model*s to pass data to a Razor view rather than expose the data as properties on themselves (as Razor Pages do with page models). This provides a clearer separation between the various "models" than in Razor Pages, though both cases use the same general MVC design pattern.

Another big difference between Razor Pages and MVC controllers is that MVC controllers typically use conventional routing, as opposed to the explicit routing used by Razor Pages. I touched on conventional routing and how it differs from explicit routing in chapter 14, but you can see it in action in this MVC application.

Conventional routing defines one or more route template patterns, which are used for all the MVC controllers in your app. The default route template, shown in listing 19.1, consists of three optional segments:

```
"{controller=Home}/{action=Index}/{id?}"
```

Conventional routes must describe which controller and action should run for any given request, so they must include `controller` and `action` route parameters at a minimum. When a request is received, ASP.NET Core